

B⁺-Tree Deletion: Explanation and Examples

1 Introduction

Deleting a key from a B⁺-tree is more involved than insertion because it can create a **node underflow**, where a node contains fewer keys than the minimum required. To maintain the B⁺-tree properties, underfull nodes must be corrected using one of the following strategies:

- **Redistribution (Borrowing):** Transfer keys from a sibling node to restore the minimum occupancy without merging.
- **Merging:** Combine the underfull node with a sibling and remove the corresponding separator key from the parent.

Deletion can propagate upward:

- If a merge causes the parent to become underfull, it may need redistribution or merging as well.
- In the worst case, the root may lose keys, potentially decreasing the tree height by one level.

Despite its complexity, the deletion operation preserves the **balance** of the tree and guarantees logarithmic performance for searches, insertions, and deletions.

Step 1: Initial Tree Before Deletion

The tree before deleting “Srinivasan”:

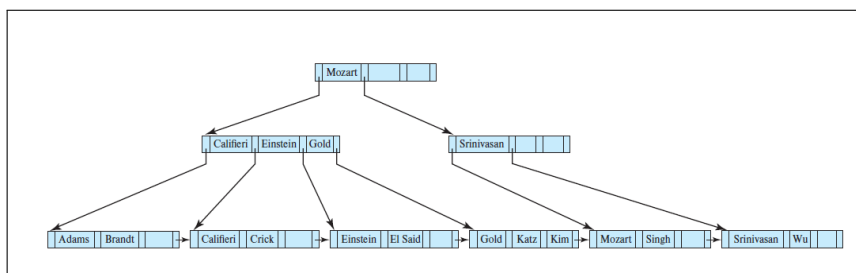


Figure 1: Initial B⁺-tree before deletion of “Srinivasan”

Step 2: Deleting “Srinivasan”

Step-by-Step Deletion of “Srinivasan”

1. Locate “Srinivasan” using the lookup algorithm.
2. Remove “Srinivasan” from its leaf node.
3. Check leaf occupancy: if it has fewer than $\lceil (n-1)/2 \rceil$ keys, an underflow occurs.
4. Fix underflow by redistributing keys with a sibling and updating the parent separator key.

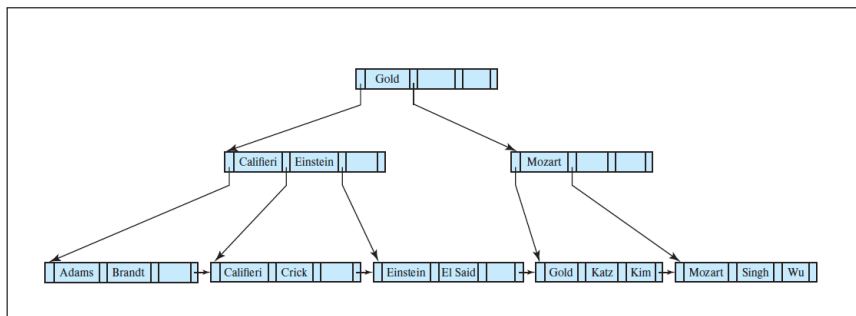


Figure 2: Deletion of “Srinivasan” with redistribution to sibling

Step 3: Deleting “Singh” and “Wu”

Deletion Details

- Deleting “Singh” does not cause underflow; no restructuring is required.
- Deleting “Wu” causes the leaf to become underfull:
 - Redistribute keys from sibling (e.g., move “Kim” between nodes).
 - Update parent separator key accordingly.

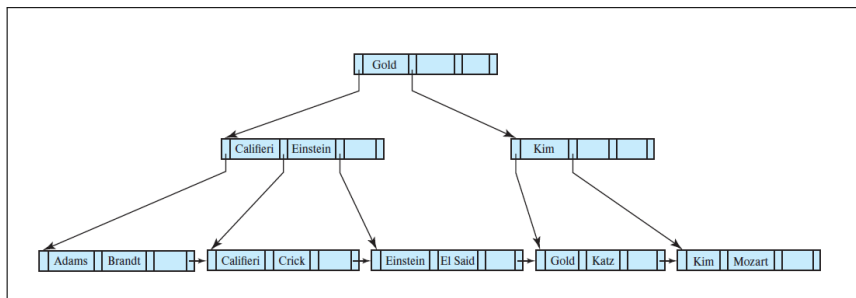


Figure 3: Deletion of “Singh” and “Wu” with redistribution

Step 4: Deleting “Gold”

Merge Example

1. Delete “Gold” from its leaf node.
2. Leaf becomes underfull; merge with sibling.
3. Remove the separator key from parent.
4. If parent becomes underfull, apply merging recursively up to the root.
5. If root ends up with a single child, reduce tree height by making that child the new root.

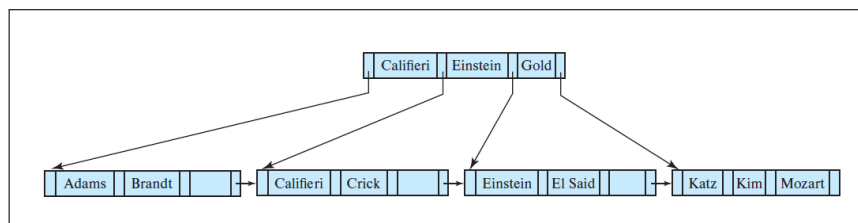


Figure 4: Deletion of “Gold” and merging of nodes

General Deletion Algorithm

Algorithm: Deletion from B^+ -Tree

procedure delete(key K):

1. Locate leaf L containing K .
2. Remove K from L .
3. If L has enough keys, stop.
4. Otherwise:
 - 4.1. Try to redistribute keys with a sibling.
 - 4.2. If redistribution is not possible, merge with sibling and remove the separator from parent.
5. If parent becomes underfull, recursively apply deletion up to root.
6. If root has only one child and becomes empty, make the child the new root.

Key Insights

- Deletion may cause **redistribution** or **merging**, but the tree always remains balanced.
- Height may decrease if the root has only one child after merging.
- Ensures all B⁺-tree properties are preserved.
- Worst-case complexity: $O(\log N)$.